Methodology

We implement a **multi-stage RAG pipeline** in which document retrieval, generation, verification, and correction are handled by specialized modules. Each stage uses state-of-the-art techniques and is informed by prior research. The overall workflow is orchestrated as an agentic loop (via LangGraph) that iteratively refines the output until it is fully grounded in the source documents[1][2]. The main stages are:

- **Document Ingestion and Chunking.** Unstructured PDFs are processed with PyMuPDF to extract text. We perform cleaning (removing headers/footers and fixing hyphenation) and split the text into *overlapping semantic chunks* (e.g. 500 tokens with 50-token overlap) to preserve sentence continuity across boundaries. Each chunk is encoded using a Sentence-BERT model (we use the lightweight `all-MiniLM-L6-v2`) to produce a dense embedding[3]. This approach (SBERT) is known to yield semantically meaningful embeddings that can be compared via cosine similarity very efficiently[3]. Chunk embeddings and metadata (document ID, page, chunk index) are stored in a FAISS-backed vector index (ChromaDB) for fast retrieval. (Encoding each chunk with a sentence-transformer ensures that semantically similar passages retrieve accurately with minimal compute[3].)

- **Semantic Retrieval.** At query time, the user's query is encoded using the same embedding model. A dense similarity search (cosine distance) over the indexed chunk embeddings returns the top-k relevant passages (we use $k=5$ by default). Dense passage retrieval has been shown to substantially outperform classical BM25 search in open-domain QA[4]. The retrieved chunks (with their IDs and metadata) form the *context* for answering. This context is passed to the next agent as both the generation prompt context and as evidence for later verification.

- **Generation Agent.** We use a **Llama-3-8B-Instruct** model (served via Ollama or Groq for quantized inference) as the primary answer generator. This agent receives a structured prompt that includes (a) the user's query and (b) all retrieved context chunks. The system prompt instructs the model to *"Answer strictly based on the provided context. Do not introduce any information not in the context passages."* In practice, we enforce output in standalone sentences, each stating one claim. (This mirrors the RAG setup in Lewis et al. (2020), which concatenates retrieved passages to the prompt so the LLM grounds its answer in external text[5].) We rely on the model to produce a fluent answer draft, but we do **not** assume it will automatically be fully faithful.

- **Verification Agent (NLI Fact-Checker).** Each sentence in the generated draft is independently checked against the retrieved context using a pretrained NLI model. We use the HuggingFace `cross-encoder/nli-deberta-base`, a DeBERTa-based classifier fine-tuned on MNLI and related entailment data. DeBERTa models achieve state-of-the-art results on NLI benchmarks[6], making them well-suited for sentence-level entailment. For each sentence (the hypothesis) we compute entailment scores with respect to each retrieved chunk (the premise). We interpret a high entailment probability as evidence that the sentence is supported by at least one chunk. Concretely, if **max entailment score ≥ 0.80**, we mark the sentence as *verified* and record the chunk that gave the highest score. Otherwise we flag it. (Thresholds around 0.7–0.8 are typical in NLI-based fact-checking[7][8].) This produces a report of `form (sentence, verdict, supporting_chunk_id)`.

Sentences marked **Entailment** are accepted; those marked **Neutral/Contradiction** are flagged for correction. (Falke et al. (2019) demonstrated that entailment models can identify unsupported claims in generated text[7], motivating this approach.)

- **Self-Correction Loop.** We embed the above generator-verifier pair in a LangGraph agent workflow with a conditional loop [1]. After the first draft is scored, the workflow checks: "Are any sentences flagged as unsupported?" If **yes**, the workflow re-invokes the Generation Agent with an *augmented prompt* that highlights only the flagged sentences and their (lack of) evidence. For example, the prompt may say: "The following statement was *not* supported by the evidence:

$$flaggedsentence$$

. Rewrite it using only the retrieved context facts." The model then attempts to revise just those sentences. We allow up to three total iterations. This looped design is directly inspired by agentic LLM frameworks, where conditional edges and cycles let the system iteratively refine its own output [1] [9]. (Wu et al. (2023) and Wang et al. (2024) survey similar multi-agent correction approaches.) If after 3 tries a sentence still cannot be grounded, the system omits it or replaces it with a disclaimer. This fail-safe prevents infinite loops and enforces honesty. (Notably, Xu et al. (2026) also explore NLI-guided iterative search to enforce factuality, suggesting that even complex multi-hop queries benefit from this kind of loop[10].)

- **Citation UI Pipeline.** Once all sentences are verified, the final answer is assembled as follows: for each sentence marked Entailment, we know the supporting chunk ID. We emit the answer as JSON with sentence-level citations. In the React/Tailwind frontend, each sentence is rendered as a span that highlights on hover/click, showing the source chunk text and metadata (document title, page). Any unsupported sentence would appear in a warning style (though our goal is to have none). In this way the system provides *interactive, sentence-level citations* so the user can audit each claim. This approach is analogous to scholarly interfaces where claims link to references, but automated by the pipeline's internal verification results [2] .

- **Evaluation Protocol.** We will evaluate the system on held-out QA tasks using the RAGAs framework [2]. Specifically, we measure:

- *Faithfulness* (the fraction of final-answer sentences entailed by retrieved context [2]),
- *Answer Relevance* (semantic score to question), and
- *Context Recall* (coverage of ground-truth evidence in retrieved chunks) as defined in Es et al. (2024) [2] .

We also introduce **Correction Delta**: the improvement in Faithfulness after the correction loop, akin to the atomic-claim scoring of Min et al. (2023)[8]. In the baseline (single pass RAG) we expect many hallucinated sentences; after correction, we target a Faithfulness > 0.90 [2] and a Delta > 0.15 (i.e. a 15-point gain from iteration). The FActScore methodology inspires breaking answers into atomic facts and verifying each[8], so we will compute how many facts remain unsupported before vs. after correction. Overall, the evaluation directly measures how the agentic loop and NLI verifier improve factual accuracy.

Each component is implemented with open-source tools (see *Tools & Tech* below), and all prompts and agent configurations are documented. By integrating retrieval, generation, independent NLI checking, and

iterative revision, this methodology ensures that every final claim is explicitly grounded in source text[5][2]. It builds on prior RAG work (Lewis et al., 2020 [5]; Karpukhin et al., 2020 [4]) and NLI fact-checking (Falke et al., 2019[7]) but wraps them in a multi-agent cycle. This approach aims to yield reproducible, auditable answers grounded in evidence.

References for Methodology: We leverage foundational RAG methods[5] [4], sentence embeddings[3], and DeBERTa NLI models[6], and our evaluation follows the RAGAS framework[2]. These citations support our choice of techniques at each step.

---

[1] Building a Self-Correcting RAG Pipeline with LangGraph: A Practical Guide | by Vishnudhat Natarajan | Medium
https://medium.com/@vishnudhat/building-a-self-correcting-rag-pipeline-with-langgraph-a-practical-guide-b4add131d877

[2] RAGAs: Automated Evaluation of Retrieval Augmented Generation - ACL Anthology
https://aclanthology.org/2024.eacl-demo.16/

[3] [1908.10084] Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks
https://arxiv.org/abs/1908.10084

[4] [2004.04906] Dense Passage Retrieval for Open-Domain Question Answering
https://arxiv.org/abs/2004.04906

[5] [2005.11401] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
https://arxiv.org/abs/2005.11401

[6] [2006.03654] DeBERTa: Decoding-enhanced BERT with Disentangled Attention
https://ar5iv.labs.arxiv.org/html/2006.03654

[7] Ranking Generated Summaries by Correctness: An Interesting but Challenging Application for Natural Language Inference - ACL Anthology
https://aclanthology.org/P19-1213/

[8] FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation - ACL Anthology
https://aclanthology.org/2023.emnlp-main.741/

[9] LangGraph: Building self-correcting RAG agent
https://learnopencv.com/langgraph-self-correcting-agent-code-generation/

[10] [2604.10734] Self-Correcting RAG: Enhancing Faithfulness via MMKP Context Selection and NLI-Guided MCTS
https://arxiv.org/abs/2604.10734